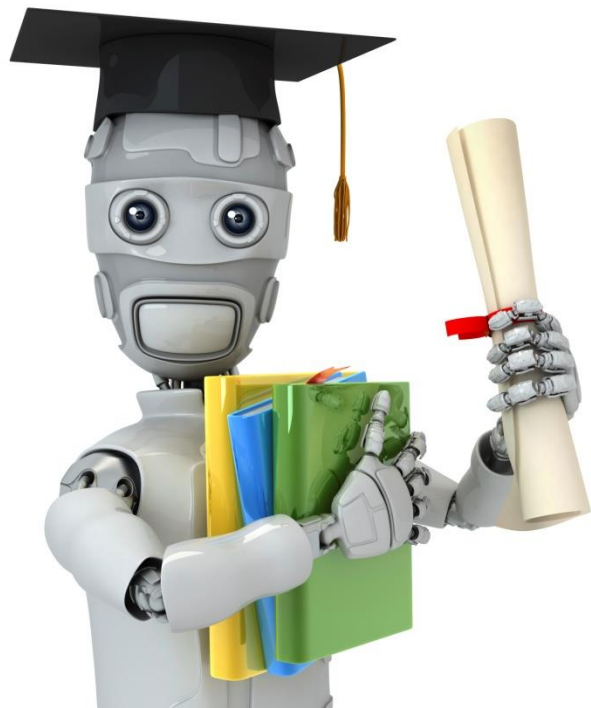


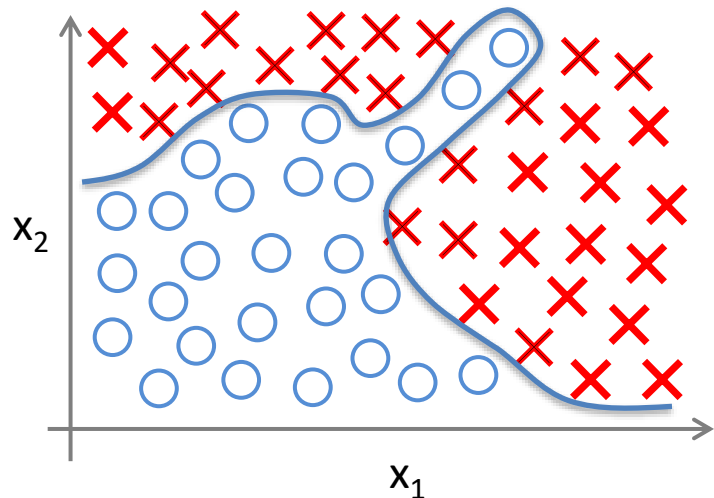


机器学习

神经网络:表述

非线性假设

非线性分类



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^3 x_2 + \theta_6 x_1 x_2^2 + \dots)$$

x_1 = 房屋面积

x_2 = 卧室数量

x_3 = 楼层

x_4 = 房龄

...

x_{100}

$$x_1 x_2 + x_1 x_3 + x_1 x_4 + \dots + x_2 x_3 + x_2 x_4 + \dots + x_{99} x_{100}$$

→ 接近 5,000 个特征

$$x_1 x_2 x_3 + x_1 x_2 x_4 + x_1 x_2 x_5 + \dots + x_2 x_3 x_4 + x_2 x_3 x_5 + \dots + x_{98} x_{99} x_{100}$$

→ 多于 170,000 个特征!!

这是什么？

你看到的是这个：



但摄像机看到的是这一点：

194	210	201	212	199	213	215	195	178	158	182	209
180	189	190	221	209	205	191	167	147	115	129	163
114	126	140	188	176	165	152	140	170	106	78	88
87	103	115	154	143	142	149	153	173	101	57	57
102	112	106	131	122	138	152	147	128	84	58	66
94	95	79	104	105	124	129	113	107	87	69	67
68	71	69	98	89	92	98	95	89	88	76	67
41	56	68	99	63	45	60	82	58	76	75	65
20	43	69	75	56	41	51	73	55	70	63	44
50	50	57	69	75	75	73	74	53	68	59	37
72	59	53	66	84	92	84	74	57	72	63	42
67	61	58	65	75	78	76	73	59	75	69	50

计算机视觉：汽车检测



汽车

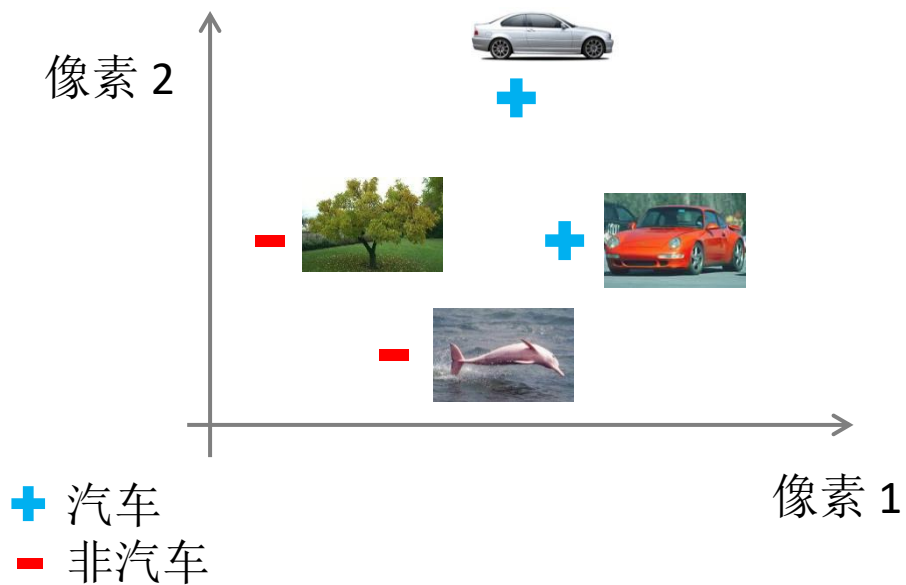
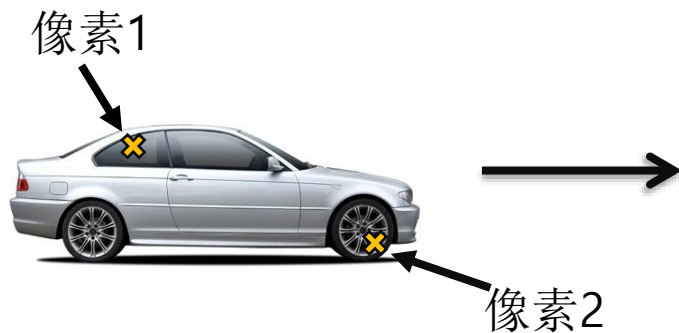


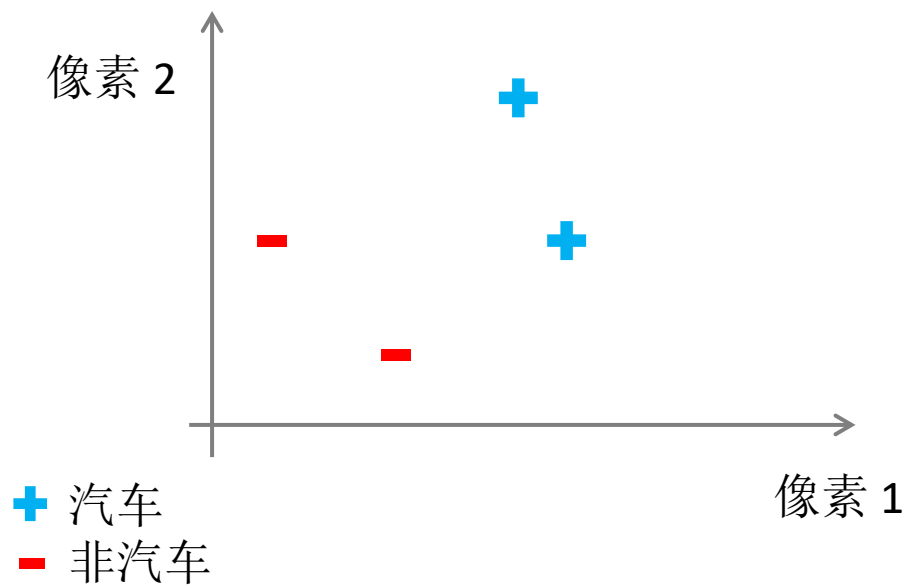
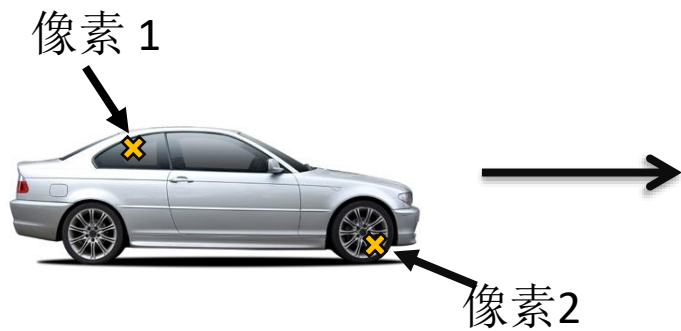
非汽车

测试:



这是什么?



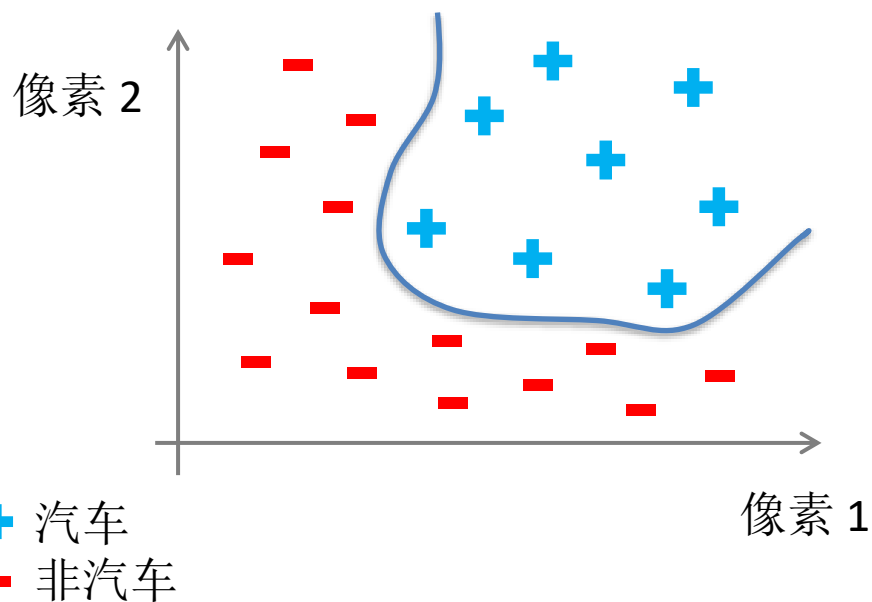


像素1



像素2

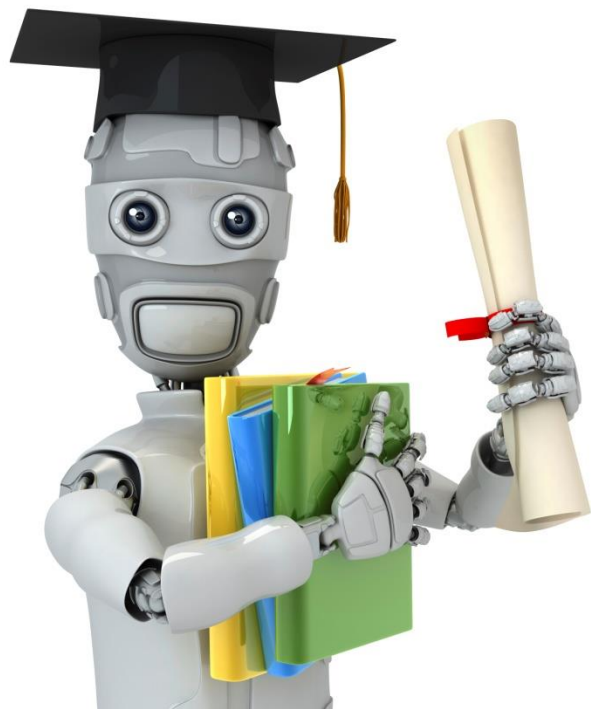
学习算法



50 x 50 像素图像 $\rightarrow$ 2500 像素
 $n = 2500$ (如果是RGB 图像, 则为7500)

$$x = \begin{bmatrix} \text{pixel 1 intensity} \\ \text{pixel 2 intensity} \\ \vdots \\ \text{pixel 2500 intensity} \end{bmatrix}$$

二次特征($x_i \times x_j$): ≈ 3 百万个特征



机器学习

神经网络: 表述

神经元与大脑

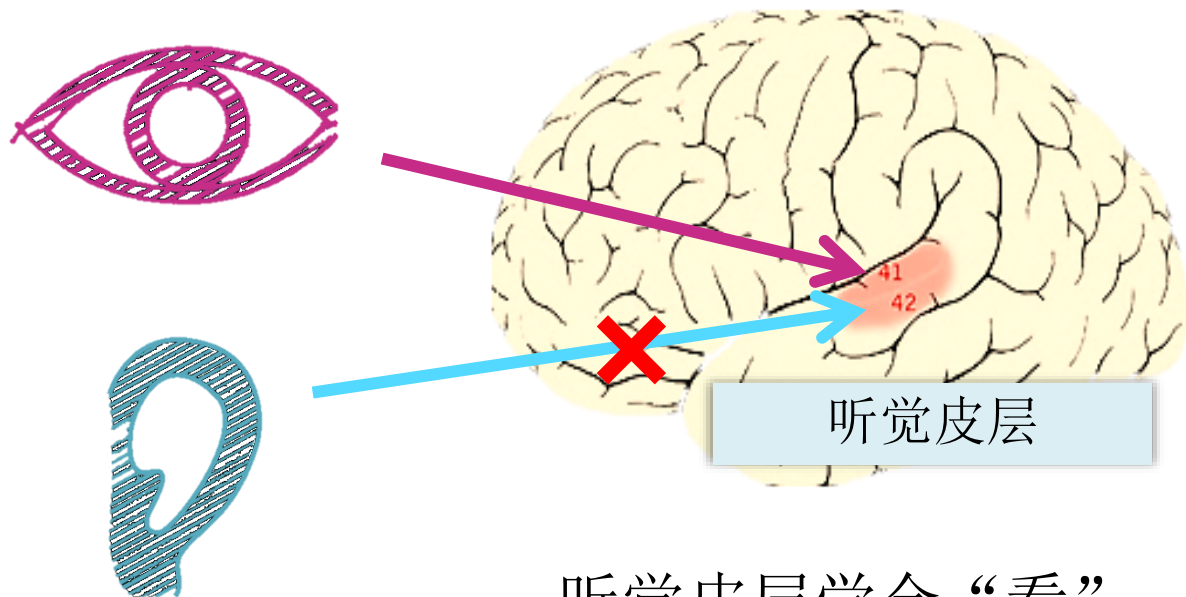
神经网络

起源：试图模仿大脑的算法。

在80年代和90年代初被广泛使用；在90年代末受欢迎度减少。

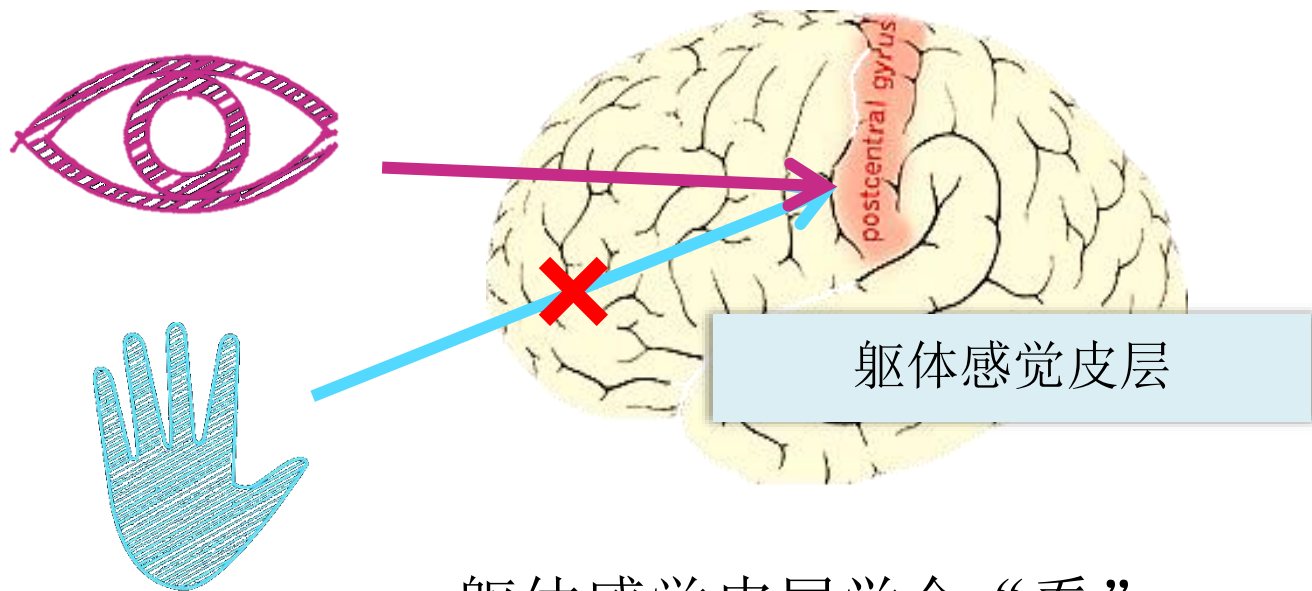
最近再次兴起：成为许多应用的先进技术。

“单一学习算法”假设



听觉皮层学会“看”

“单一学习算法”假设



躯体感觉皮层学会“看”

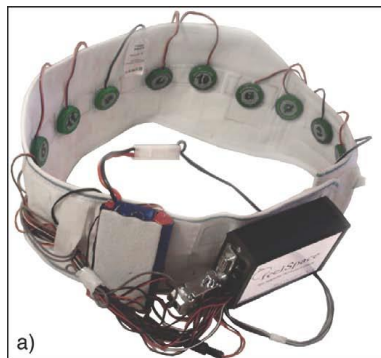
大脑中的传感器表示



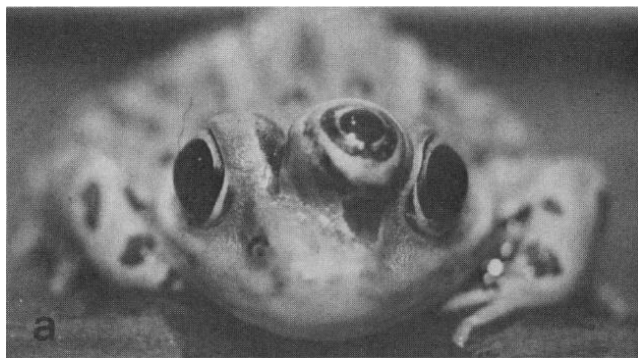
用舌头看



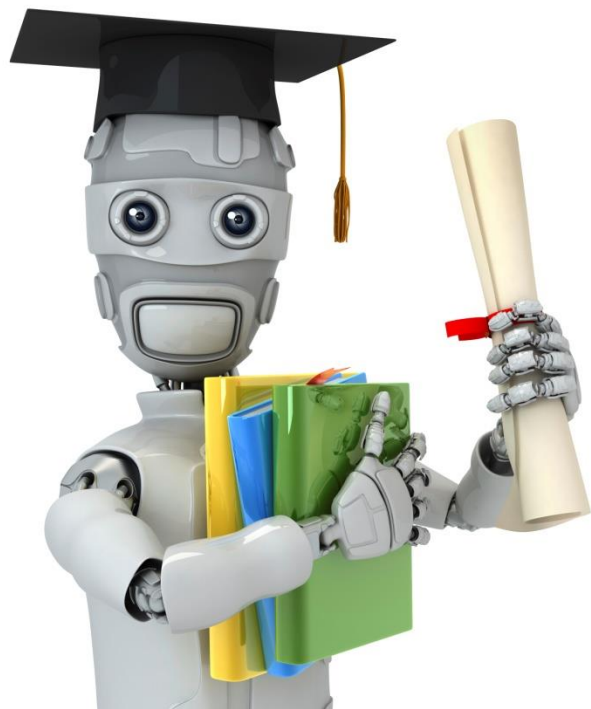
人体回声定位（声纳）



触觉带：方向感



植入第三只眼睛

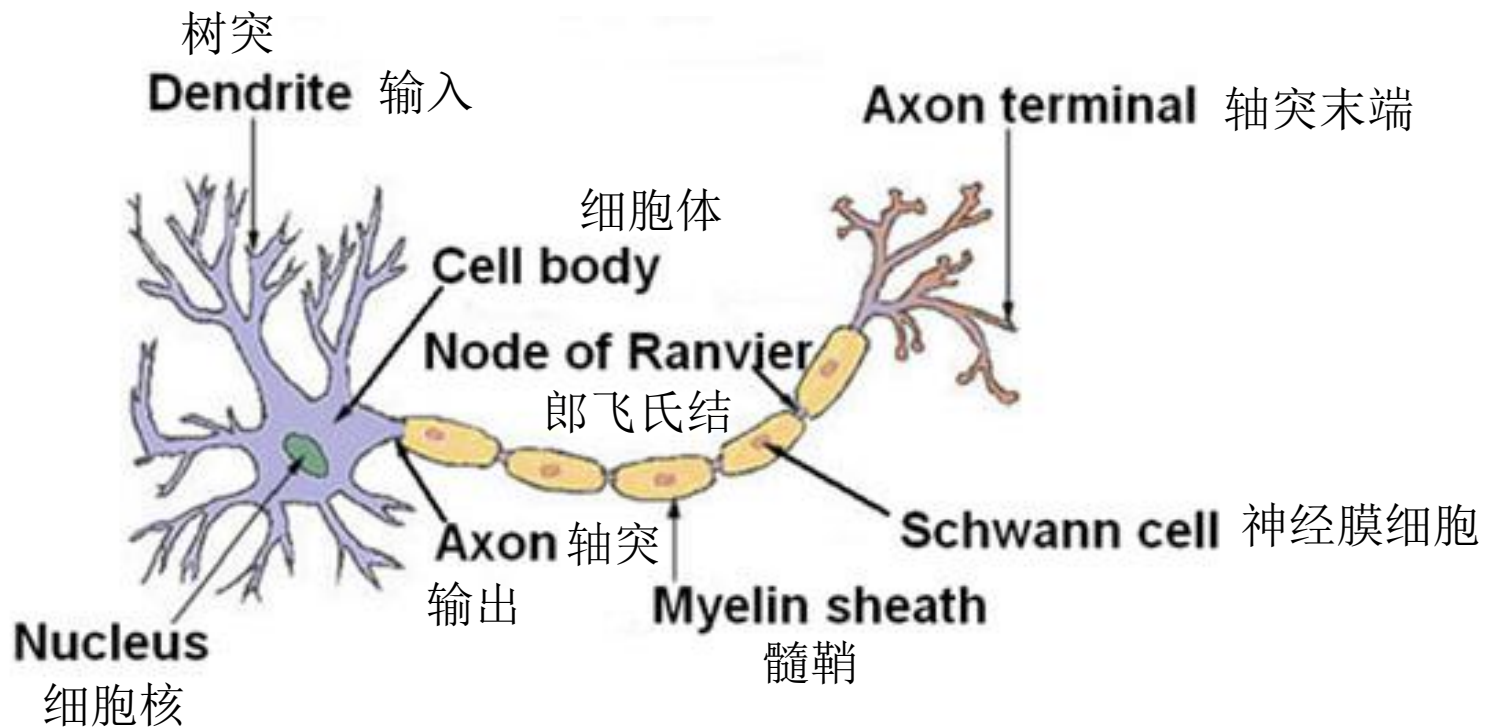


机器学习

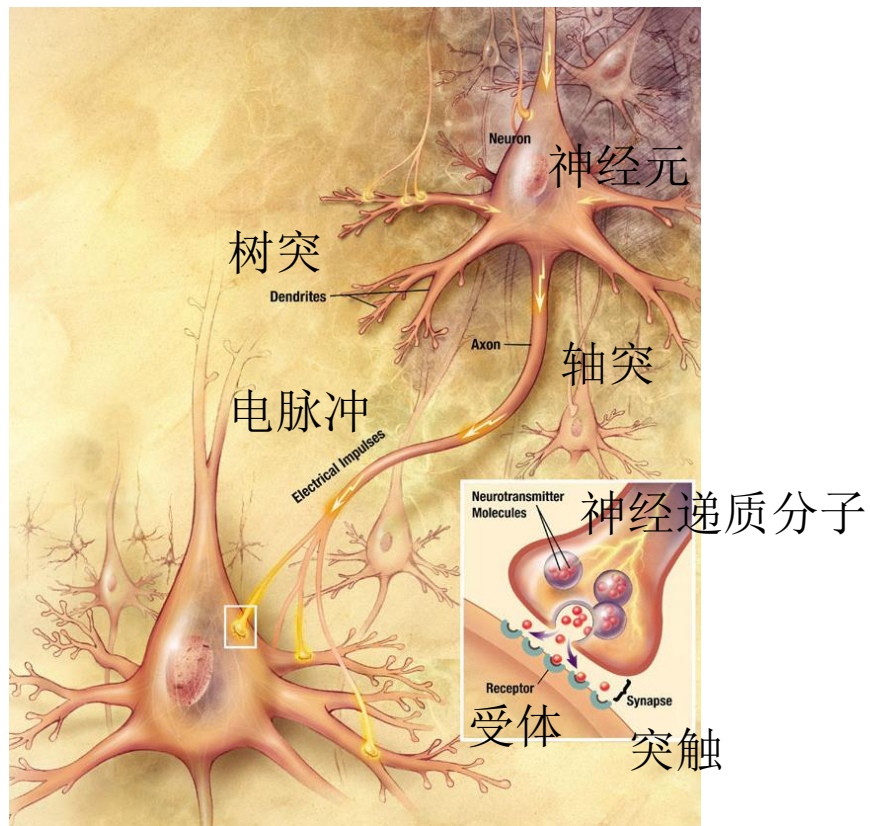
神经网络：表述

模型表示 I

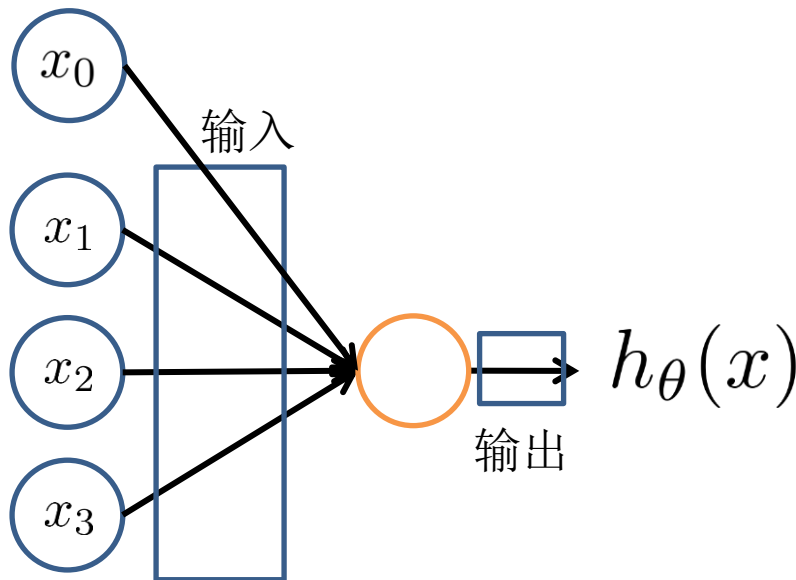
大脑中的神经元



大脑中的神经元



神经元模型：逻辑单元

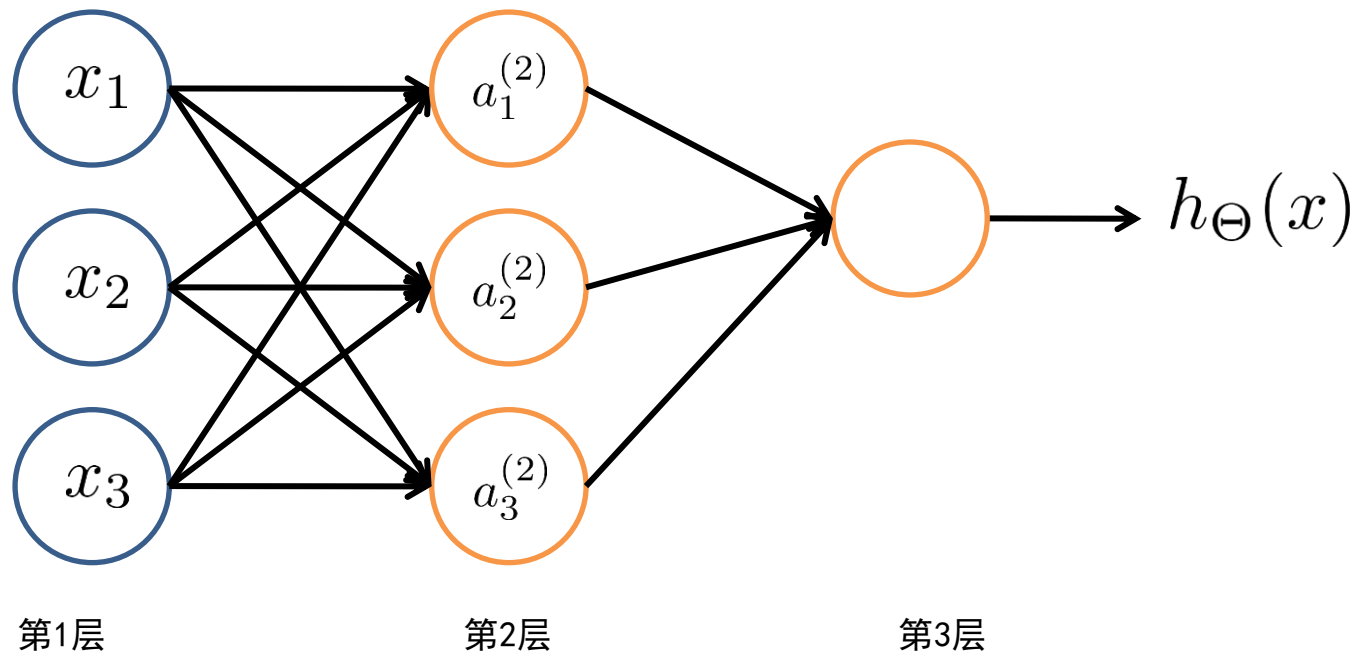


$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

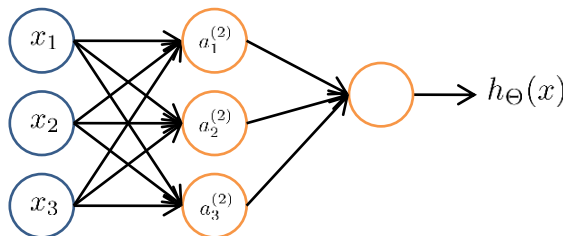
$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Sigmoid (逻辑) 激活函数.

神经网络



神经网络



$a_i^{(j)}$ = 第 j 层的第 i 个激活单元

$\Theta^{(j)}$ = 从第 j 层映射到第 $j+1$ 层时的权重矩阵

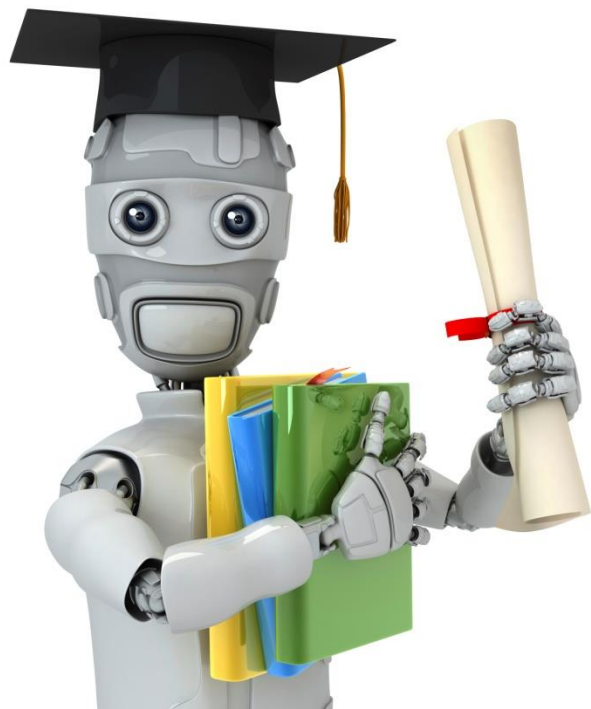
$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

如果在第 j 层有 s_j 个激活单元，在第 $j+1$ 层有 s_{j+1} 个激活单元，那么 $\Theta^{(j)}$ 的维度将是 $s_{j+1} \times (s_j + 1)$

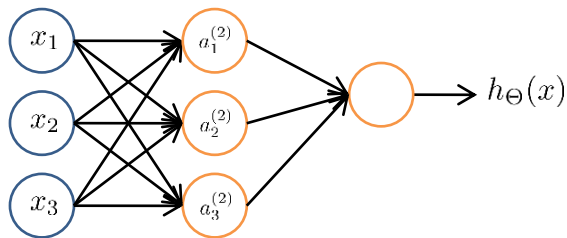


机器学习

神经网络：表述

模型表示 II

前向传播：矢量化实现



$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}$$

$$z^{(2)} = \Theta^{(1)} x$$

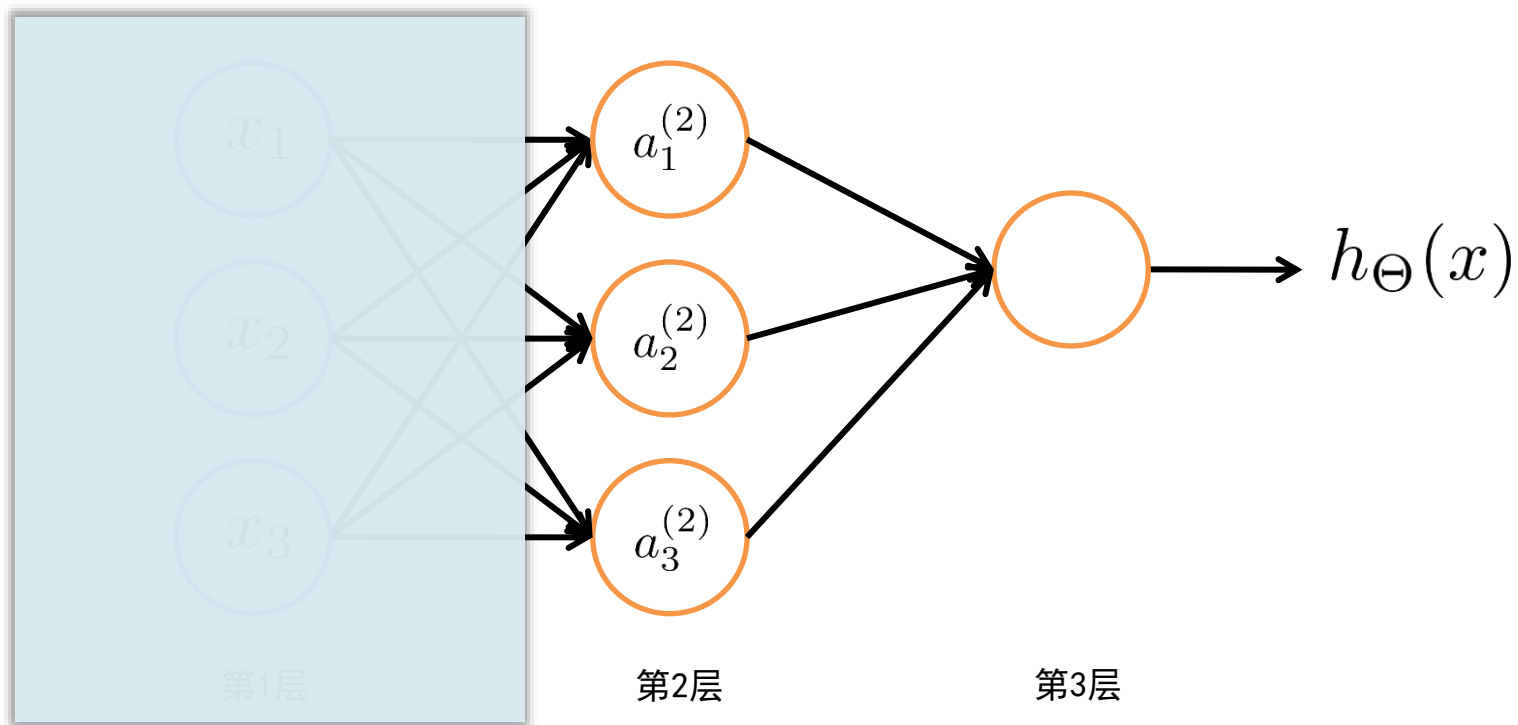
$$a^{(2)} = g(z^{(2)})$$

添加 $a_0^{(2)} = 1$.

$$z^{(3)} = \Theta^{(2)} a^{(2)}$$

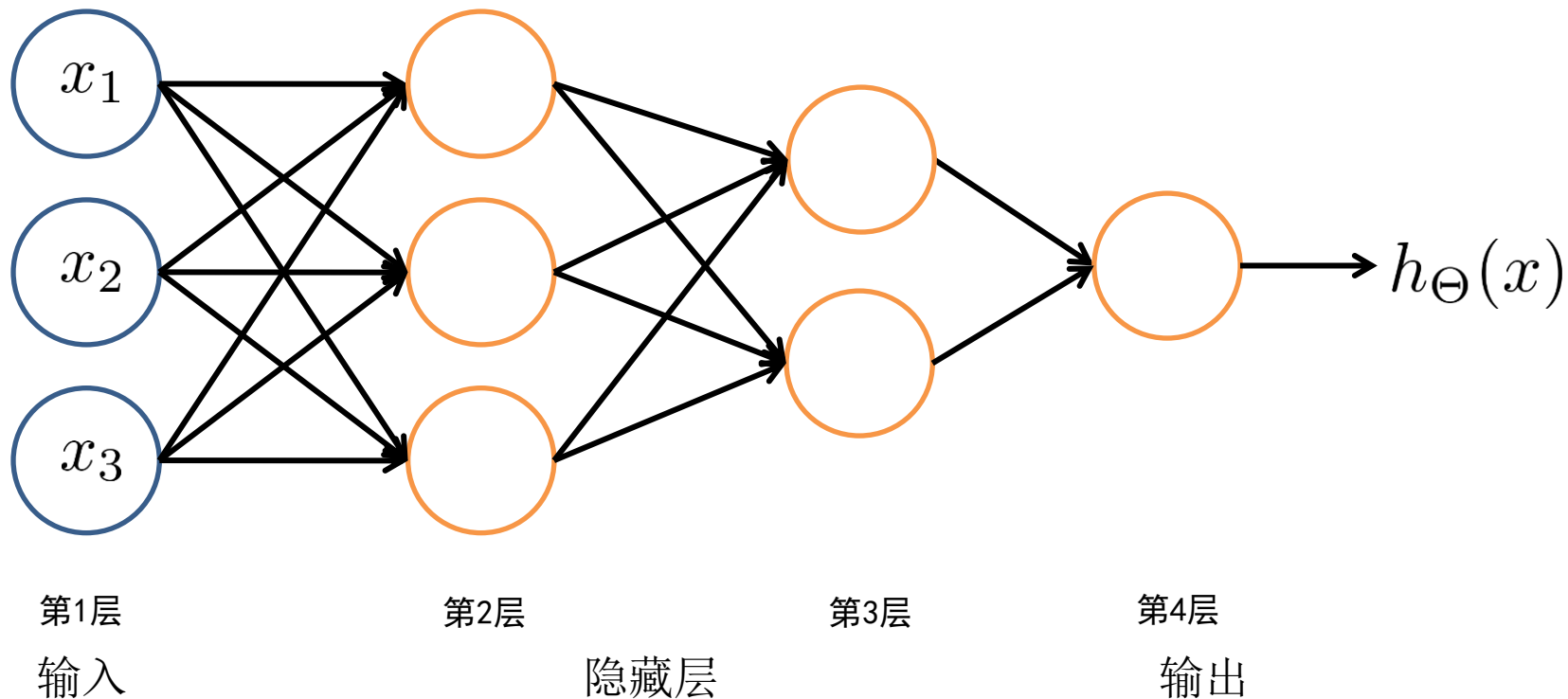
$$h_{\Theta}(x) = a^{(3)} = g(z^{(3)})$$

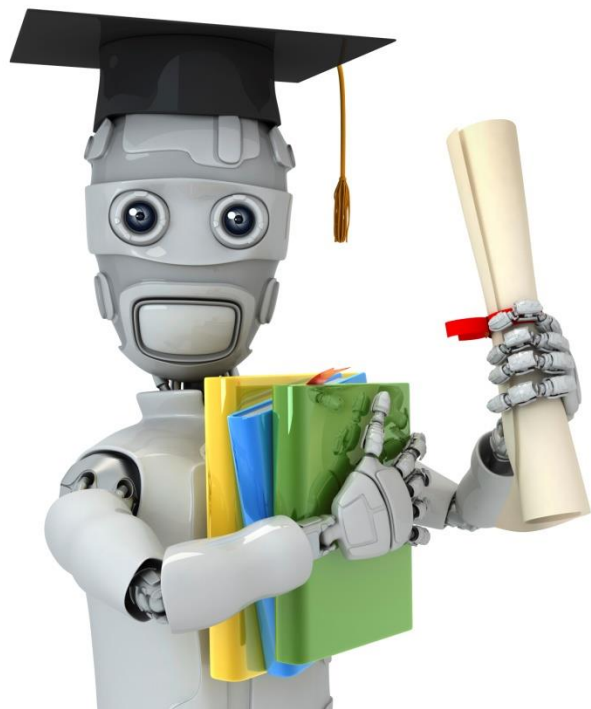
神经网络学习自身特征



$$h_{\theta}(x) = g\left(\theta_0^{(2)} a_0^{(2)} + \theta_1^{(2)} a_1^{(2)} + \theta_2^{(2)} a_2^{(2)} + \theta_3^{(2)} a_3^{(2)}\right)$$

其他网络架构





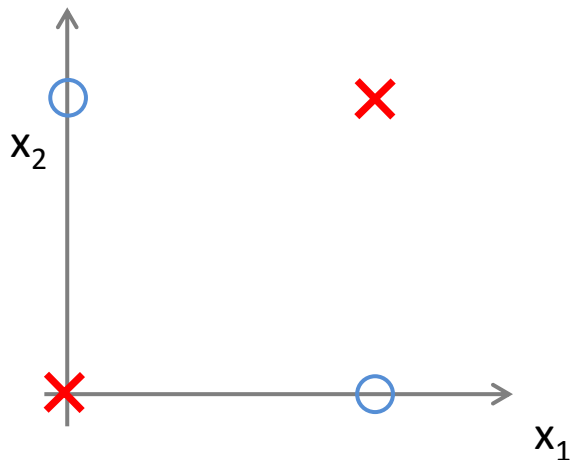
机器学习

神经网络：表述

实例和直观理解I

非线性分类示例: XOR/XNOR

x_1, x_2 是二分类(0 或 1).



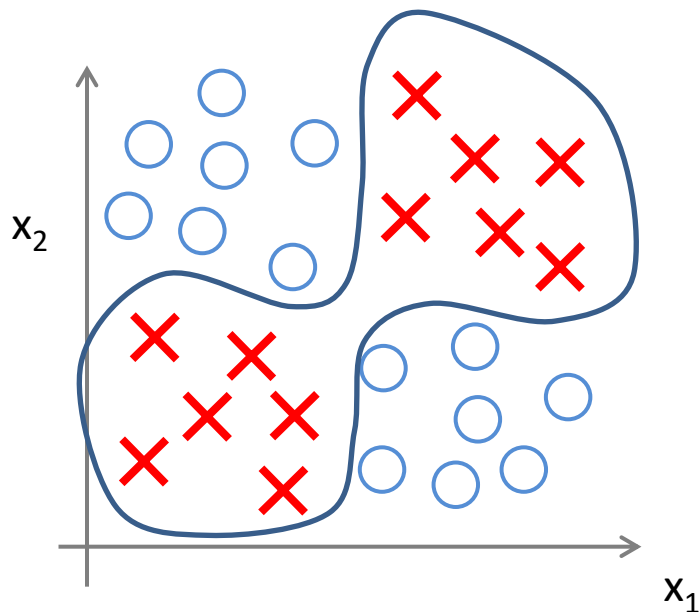
$$y = x_1 \text{ XOR } x_2$$

$$x_1 \text{ XNOR } x_2$$

$$\text{NOT } (x_1 \text{ XOR } x_2)$$

XOR真值表
XOR truth table

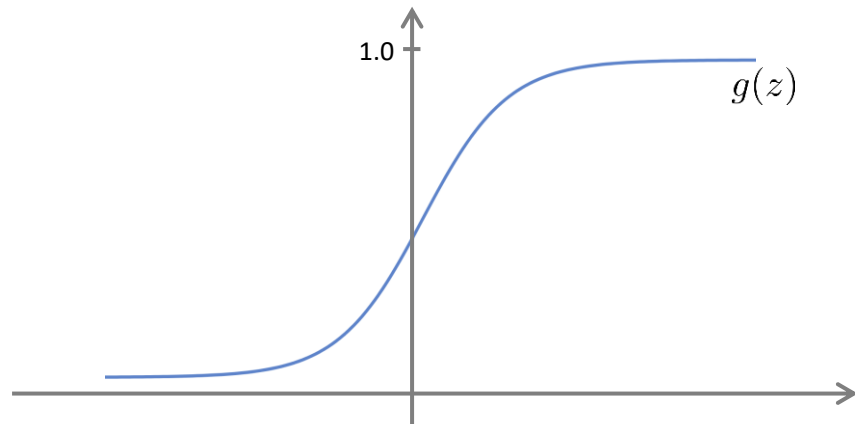
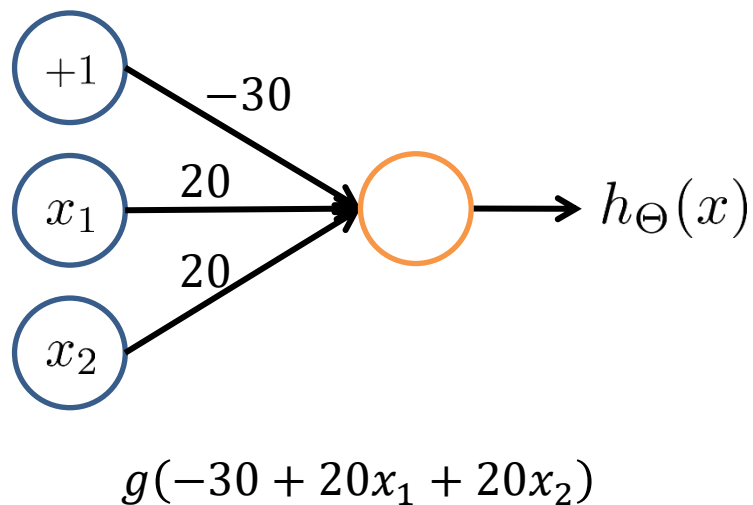
输入 Input		输出 Output
A	B	
0	0	0
0	1	1
1	0	1
1	1	0



简单例子: AND

$$x_1, x_2 \in \{0, 1\}$$

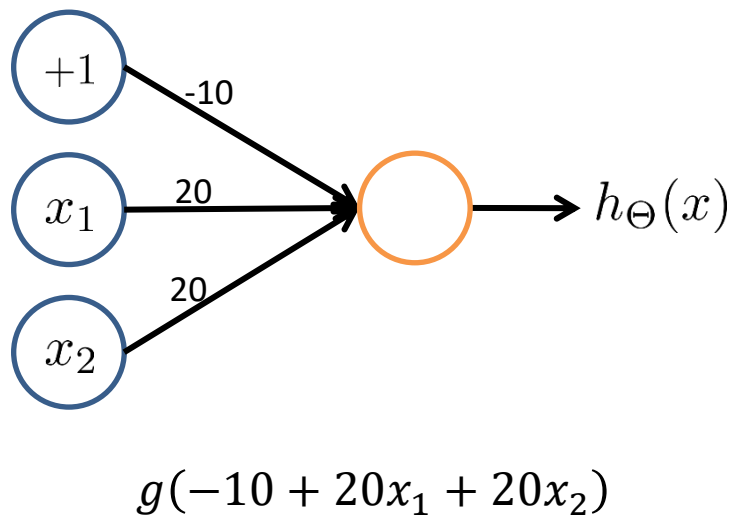
$$y = x_1 \text{ AND } x_2$$



x_1	x_2	$h_{\Theta}(x)$
0	0	≈ 0
0	1	≈ 0
1	0	≈ 0
1	1	≈ 1

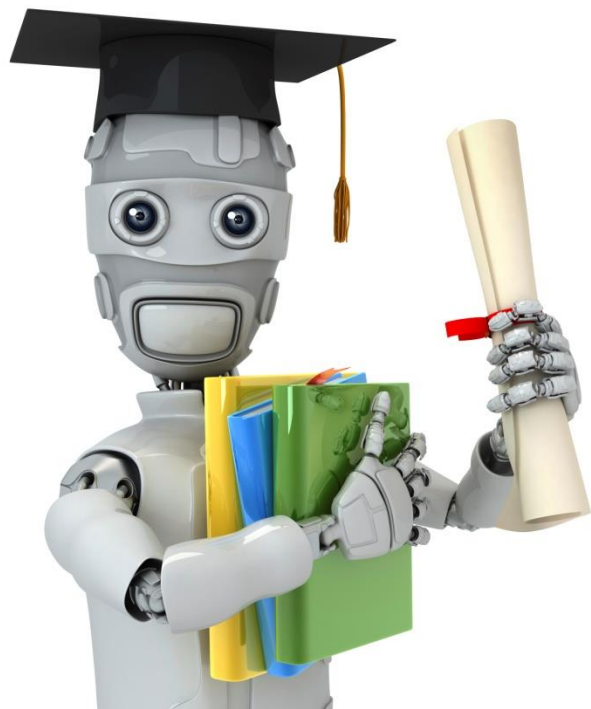
$$h_{\Theta}(x) \approx x_1 \text{ AND } x_2$$

例子: OR 函数



x_1	x_2	$h_{\Theta}(x)$
0	0	≈ 0
0	1	≈ 1
1	0	≈ 1
1	1	≈ 1

$$h_{\Theta}(x) \approx x_1 \text{ OR } x_2$$



机器学习

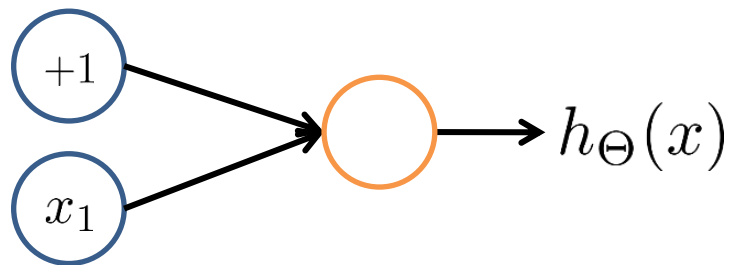
神经网络：表述

实例和直观理解 II

x_1 AND x_2

x_1 OR x_2

否定:

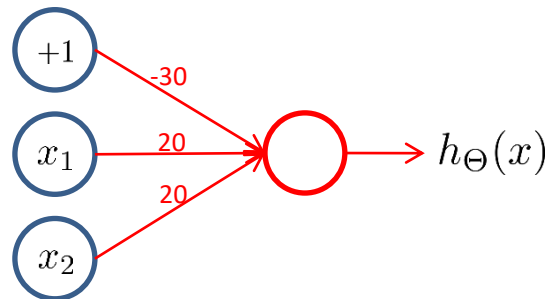


$$h_{\Theta}(x) = g(10 - 20x_1)$$

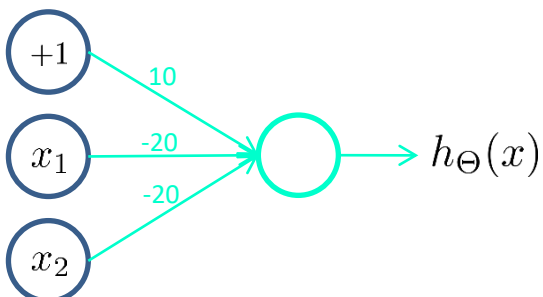
x_1	$h_{\Theta}(x)$
0	≈ 1
1	≈ 0

(NOT x_1) AND (NOT x_2)

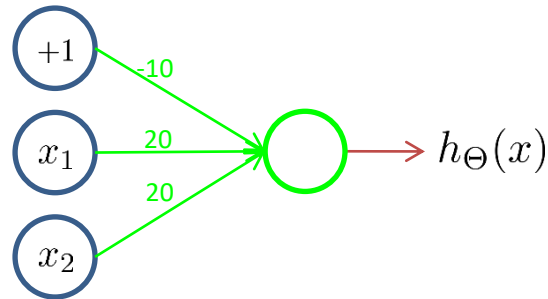
将其组合起来: x_1 XNOR x_2



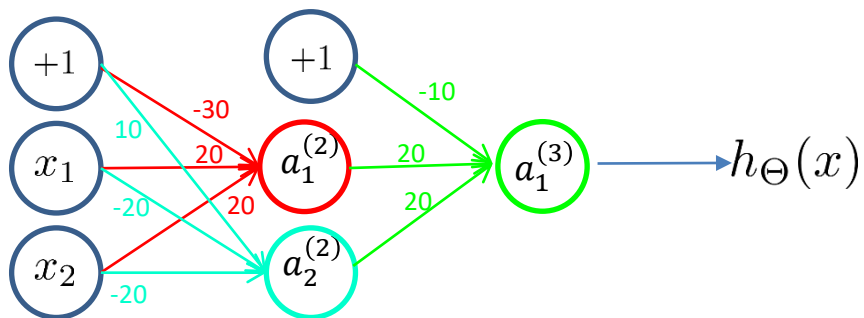
x_1 AND x_2



$(\text{NOT } x_1) \text{ AND } (\text{NOT } x_2)$

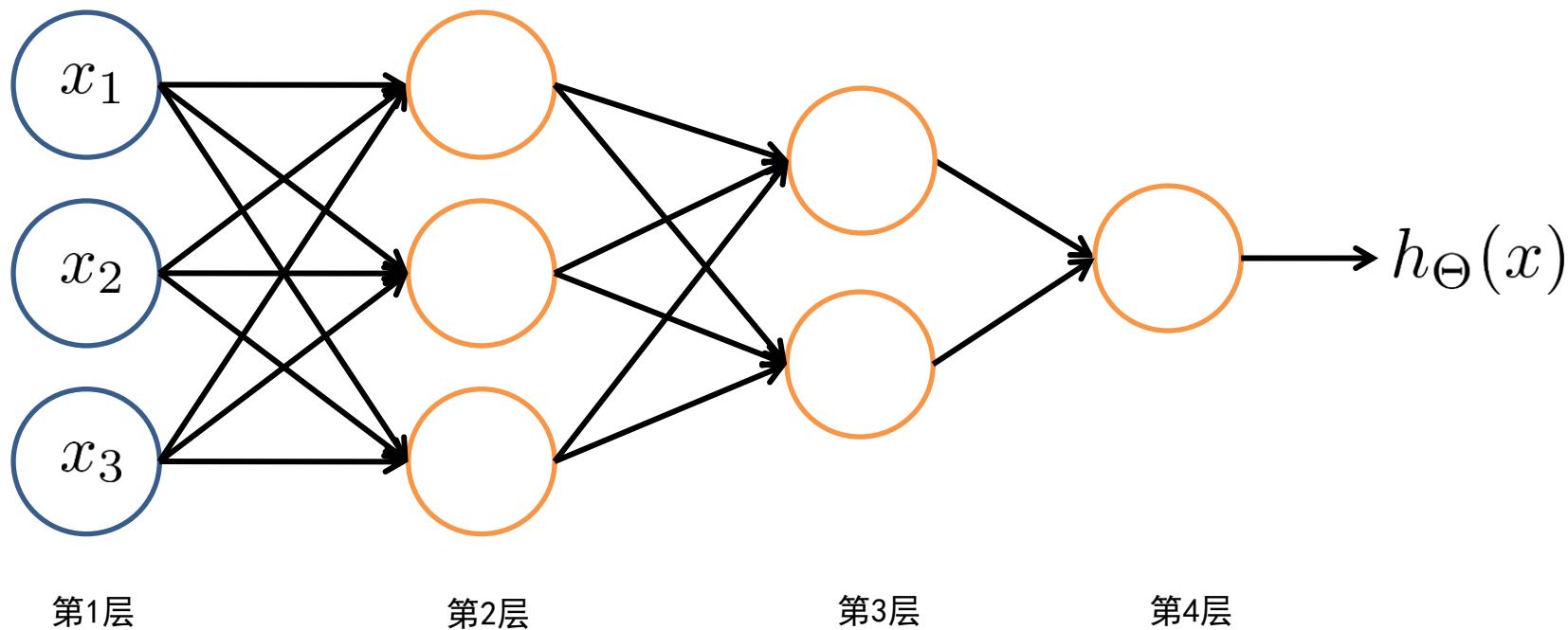


$x_1 \text{ OR } x_2$



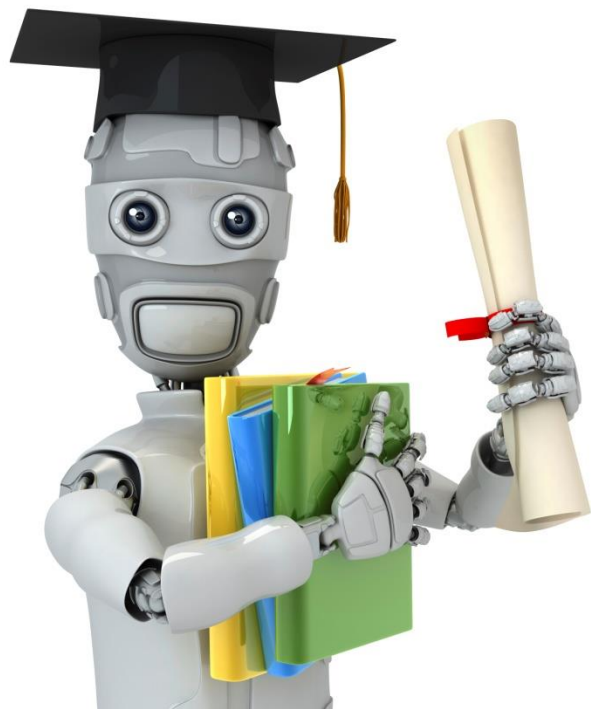
x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_{\Theta}(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

神经网络直观理解



Handwritten digit classification





机器学习

神经网络：表述

多类别分类

多个输出单元：一对多



行人



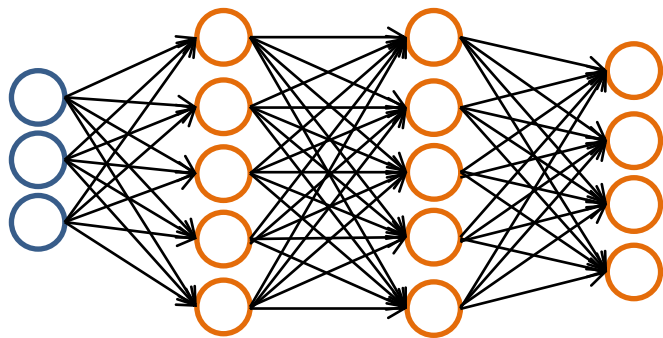
汽车



摩托车



卡车

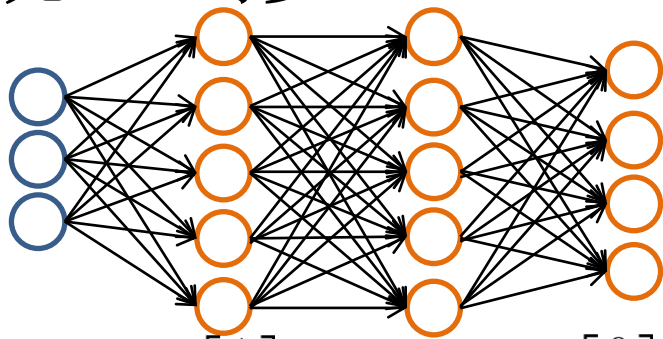


$$h_{\Theta}(x) \in \mathbb{R}^4$$

希望输出 $h_{\Theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, 等等。

行人 汽车 摩托车

多个输出单元：一对多



$$h_{\Theta}(x) \in \mathbb{R}^4$$

希望输出 $h_{\Theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, 等等。

行人

汽车

摩托车

训练集: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})$

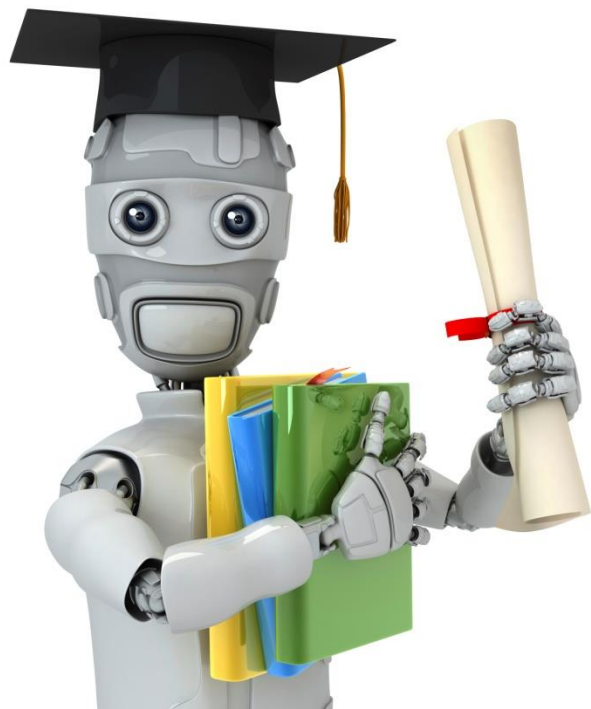
$y^{(i)}$ 是 $\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$ 中的一个。

行人

汽车

摩托车

卡车

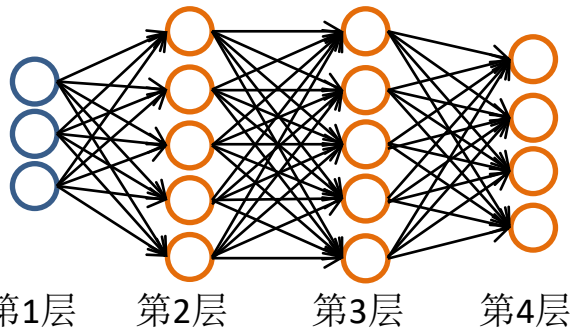


机器学习

神经网络：学习

代价函数

神经网络（分类）



二类分类

$y = 0$ or 1

1 个输出单元

$$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$$

$L =$ 网络总层数

$s_l =$ 第 l 层的神经元个数(不包括偏置单元)

多类分类(K 个类别)

$$y \in \mathbb{R}^K \quad \text{E.g.} \quad \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

行人 汽车 摩托车 卡车

K 个输出单元

代价函数

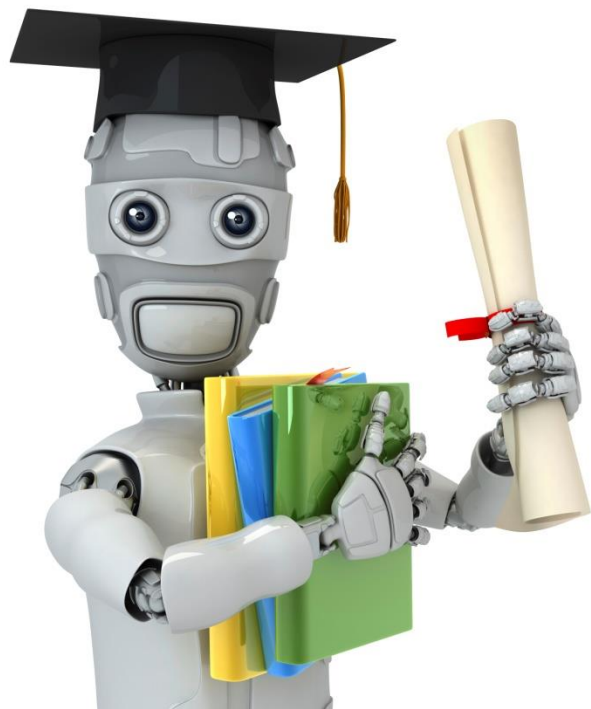
逻辑回归:

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

神经网络:

$$h_{\Theta}(x) \in \mathbb{R}^K \quad (h_{\Theta}(x))_i = i^{th} \text{ output 输出}$$

$$J(\Theta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(h_{\Theta}(x^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_{\Theta}(x^{(i)}))_k) \right] \\ + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ji}^{(l)})^2$$



机器学习

神经网络：学习

反向传播算法

梯度计算

$$J(\Theta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(h_{\Theta}(x^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_{\Theta}(x^{(i)}))_k) \right] \\ + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ji}^{(l)})^2$$

$$\min_{\Theta} J(\Theta)$$

需要代码进行计算:

- $J(\Theta)$
- $\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta)$

梯度计算

给定一个训练样本 (x, y) :

前向传播:

$$a^{(1)} = x$$

$$z^{(2)} = \Theta^{(1)} a^{(1)}$$

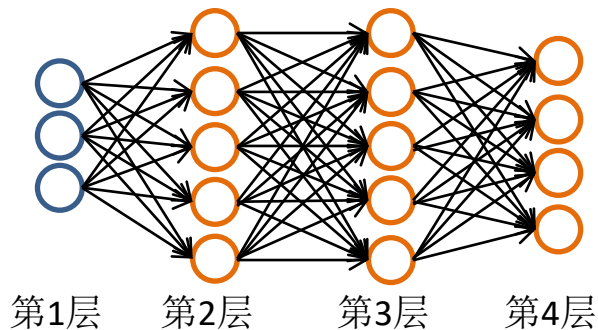
$$a^{(2)} = g(z^{(2)}) \quad (\text{add } a_0^{(2)})$$

$$z^{(3)} = \Theta^{(2)} a^{(2)}$$

$$a^{(3)} = g(z^{(3)}) \quad (\text{add } a_0^{(3)})$$

$$z^{(4)} = \Theta^{(3)} a^{(3)}$$

$$a^{(4)} = h_{\Theta}(x) = g(z^{(4)})$$



梯度计算：反向传播算法

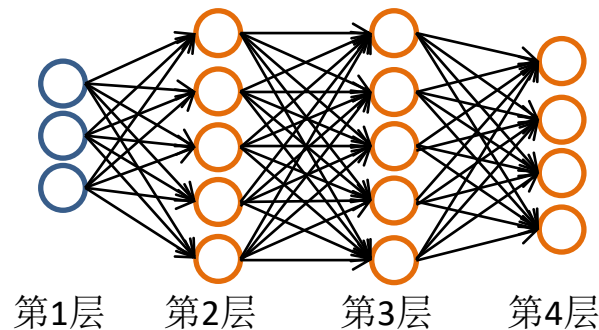
直观理解: $\delta_j^{(l)}$ = 在 l 层 j 节点的误差

对于每个输出单元(层数 $L = 4$)

$$\delta_j^{(4)} = a_j^{(4)} - y_j$$

$$\delta^{(3)} = (\Theta^{(3)})^T \delta^{(4)} \cdot * g'(z^{(3)})$$

$$\delta^{(2)} = (\Theta^{(2)})^T \delta^{(3)} \cdot * g'(z^{(2)})$$



反向传播算法

训练集 $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$

设置 $\Delta_{ij}^{(l)} = 0$ (对于所有的 l, i, j).

For $i = 1$ to m m个样本

设置 $a^{(1)} = x^{(i)}$

执行前向传播, 计算 $a^{(l)}$ for $l = 2, 3, \dots, L$

使用 $y^{(i)}$, 计算 $\delta^{(L)} = a^{(L)} - y^{(i)}$

计算 $\delta^{(L-1)}, \delta^{(L-2)}, \dots, \delta^{(2)}$

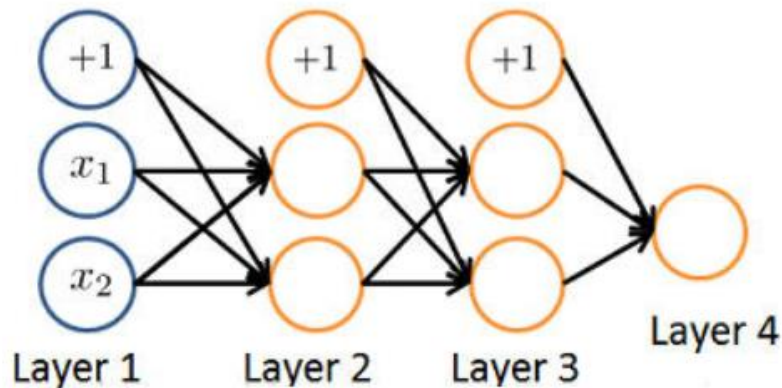
$$\Delta_{ij}^{(l)} := \Delta_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$$

$$D_{ij}^{(l)} := \frac{1}{m} \Delta_{ij}^{(l)} + \lambda \Theta_{ij}^{(l)} \text{ if } j \neq 0$$

$$D_{ij}^{(l)} := \frac{1}{m} \Delta_{ij}^{(l)} \text{ if } j = 0$$

$$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) = D_{ij}^{(l)}$$

考虑下面给出的神经网络。下列哪个方程正确地计算了 $a_1^{(3)}$ 的激活？注： $g(z)$ 是sigmoid激活函数



A. $a_1^{(3)} = g(\Theta_{1,0}^{(2)} a_0^{(2)} + \Theta_{1,1}^{(2)} a_1^{(2)} + \Theta_{1,2}^{(2)} a_2^{(2)})$

B. $a_1^{(3)} = g(\Theta_{1,0}^{(1)} a_0^{(1)} + \Theta_{1,1}^{(1)} a_1^{(1)} + \Theta_{1,2}^{(1)} a_2^{(1)})$

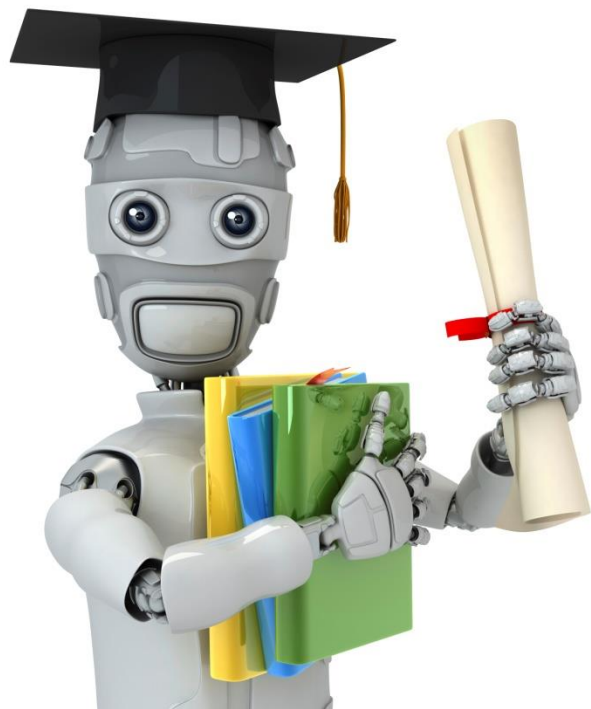
C. $a_1^{(3)} = g(\Theta_{1,0}^{(1)} a_0^{(2)} + \Theta_{1,1}^{(1)} a_1^{(2)} + \Theta_{1,2}^{(1)} a_2^{(2)})$

D. 此网络中不存在激活 $a_1^{(3)}$

您正在训练一个三层神经网络，希望使用反向传播来计算代价函数的梯度。在反向传播算法中，其中一个步骤是更新

$\Delta_{ij}^{(2)} := \Delta_{ij}^{(2)} + \delta_i^{(3)} * (a^{(2)})_j$ 对于每个 i, j ，下面哪一个是这个步骤的正确矢量化？

- A. $\Delta^{(2)} := \Delta^{(2)} + (a^{(2)})^T * \delta^{(3)}$ B. $\Delta^{(2)} := \Delta^{(2)} + (a^{(3)})^T * \delta^{(2)}$
C. $\Delta^{(2)} := \Delta^{(2)} + \delta^{(3)} * (a^{(2)})^T$ D. $\Delta^{(2)} := \Delta^{(2)} + \delta^{(3)} * (a^{(3)})^T$

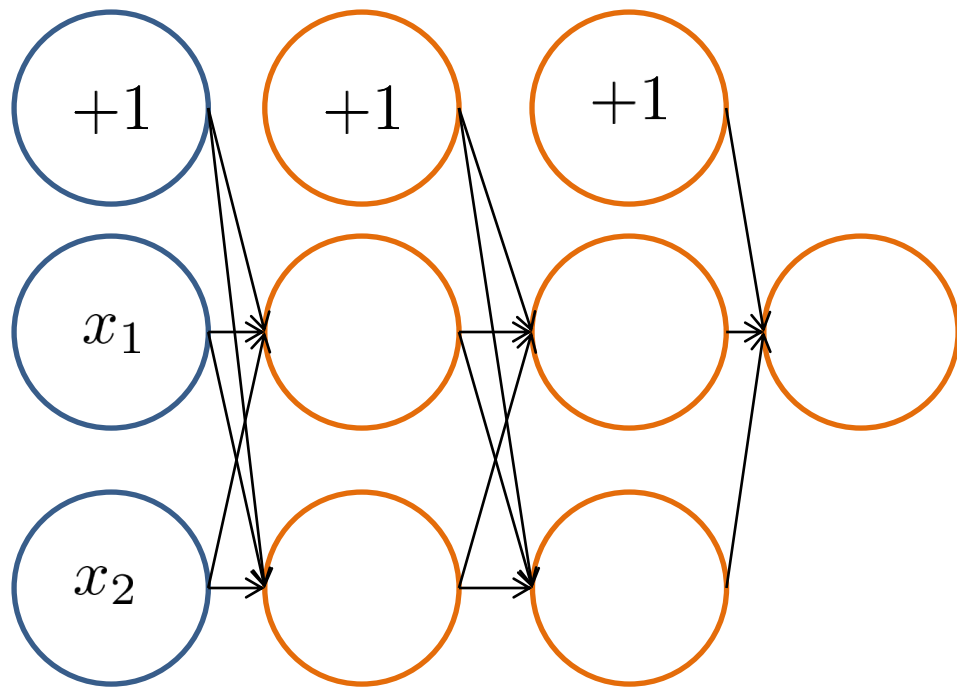


机器学习

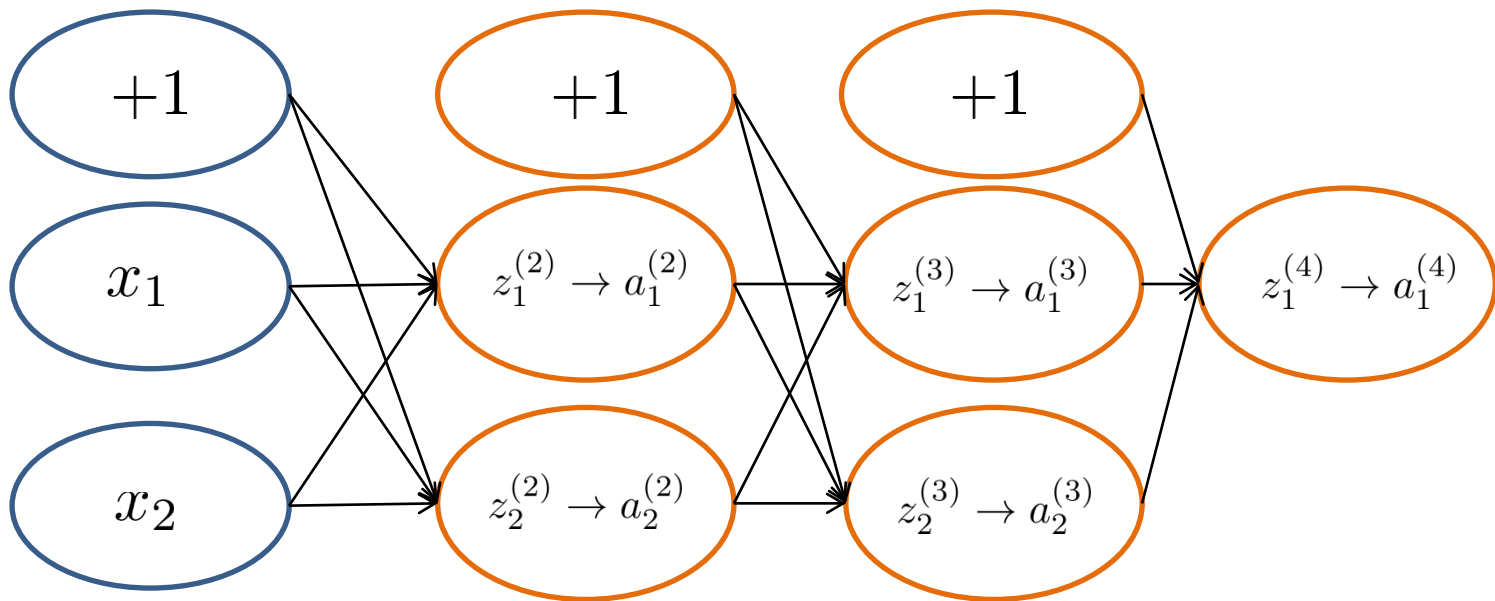
神经网络：学习

反向传播算法的 直观理解

前向传播



前向传播



反向传播算法做的是？

$$J(\Theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log(h_{\Theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - (h_{\Theta}(x^{(i)}))) \right] \\ + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ji}^{(l)})^2$$

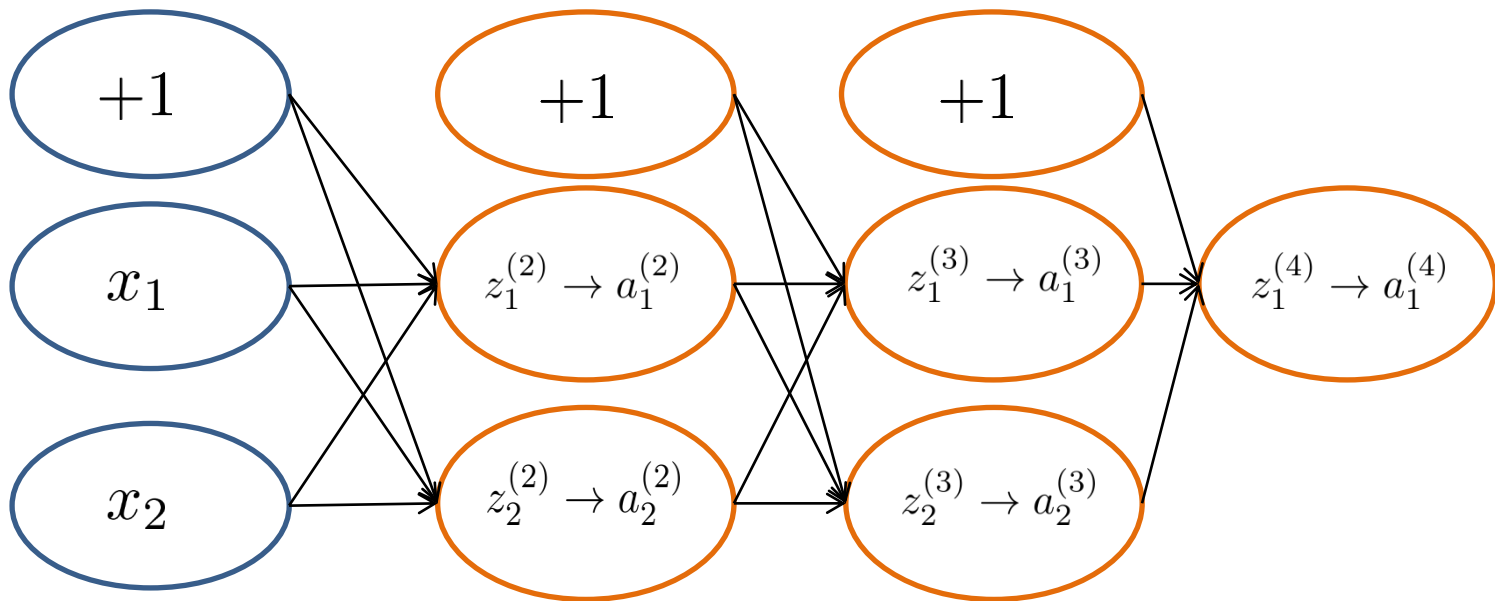
对于一个例子 $x^{(i)}$, $y^{(i)}$, 有一个输出单元, 并忽略正则化($\lambda = 0$),

$$\text{cost}(i) = y^{(i)} \log h_{\Theta}(x^{(i)}) + (1 - y^{(i)}) \log h_{\Theta}(x^{(i)})$$

(想一想 $\text{cost}(i) \approx (h_{\Theta}(x^{(i)}) - y^{(i)})^2$)

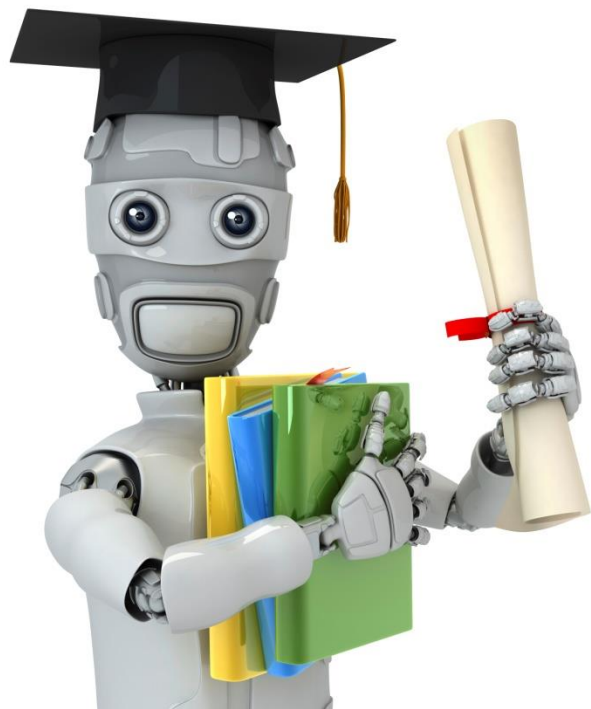
也就是说, 网络在示例 i 上的表现如何?

前向传播



$\delta_j^{(l)} = a_j^{(l)}$ 的成本“误差”(l 层的第 j 个单元).

正式地, $\delta_j^{(l)} = \frac{\partial}{\partial z_j^{(l)}} \text{cost}(i)$ (对于 $j \geq 0$), 当
 $\text{cost}(i) = y^{(i)} \log h_{\Theta}(x^{(i)}) + (1 - y^{(i)}) \log h_{\Theta}(x^{(i)})$



机器学习

神经网络：学习

实现说明：展开参数

高级优化

```
function [jVal, gradient] = costFunction(theta)
```

```
...
```

```
optTheta = fminunc(@costFunction, initialTheta, options)
```

神经网络(L=4):

$\Theta^{(1)}, \Theta^{(2)}, \Theta^{(3)}$ - matrices (**Theta1, Theta2, Theta3**)

$D^{(1)}, D^{(2)}, D^{(3)}$ - matrices (**D1, D2, D3**)

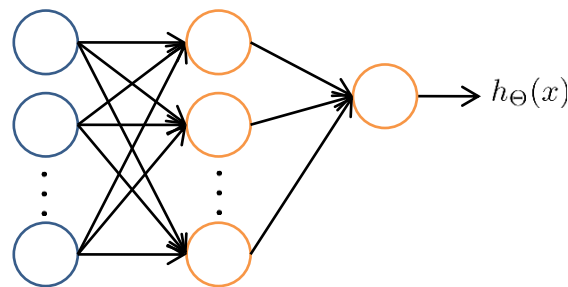
“展开”成向量

例子

$$s_1 = 10, s_2 = 10, s_3 = 1$$

$$\Theta^{(1)} \in \mathbb{R}^{10 \times 11}, \Theta^{(2)} \in \mathbb{R}^{10 \times 11}, \Theta^{(3)} \in \mathbb{R}^{1 \times 11}$$

$$D^{(1)} \in \mathbb{R}^{10 \times 11}, D^{(2)} \in \mathbb{R}^{10 \times 11}, D^{(3)} \in \mathbb{R}^{1 \times 11}$$



```
thetaVec = [ Theta1(:) ; Theta2(:) ; Theta3(:) ] ;
```

```
DVec = [ D1(:) ; D2(:) ; D3(:) ] ;
```

```
Theta1 = reshape(thetaVec(1:110) , 10 , 11) ;
```

```
Theta2 = reshape(thetaVec(111:220) , 10 , 11) ;
```

```
Theta3 = reshape(thetaVec(221:231) , 1 , 11) ;
```

学习算法

具有初始参数 $\Theta^{(1)}, \Theta^{(2)}, \Theta^{(3)}$

展开后得到 `initialTheta` 传递给

`fminunc(@costFunction, initialTheta, options)`

`function [jval, gradientVec] = costFunction(thetaVec)`

从 `thetaVec`, 得到 $\Theta^{(1)}, \Theta^{(2)}, \Theta^{(3)}$.

使用 forward prop/back prop 计算 $D^{(1)}, D^{(2)}, D^{(3)}$ 和 $J(\Theta)$.

展开 $D^{(1)}, D^{(2)}, D^{(3)}$ 得到 `gradientVec`.

假设Theta1是一个5x3矩阵, Theta2是一个4x6矩阵。令:

`thetaVec=[Theta1(:);Theta2(:)]`。下列哪一项可以正确地还原Theta2?

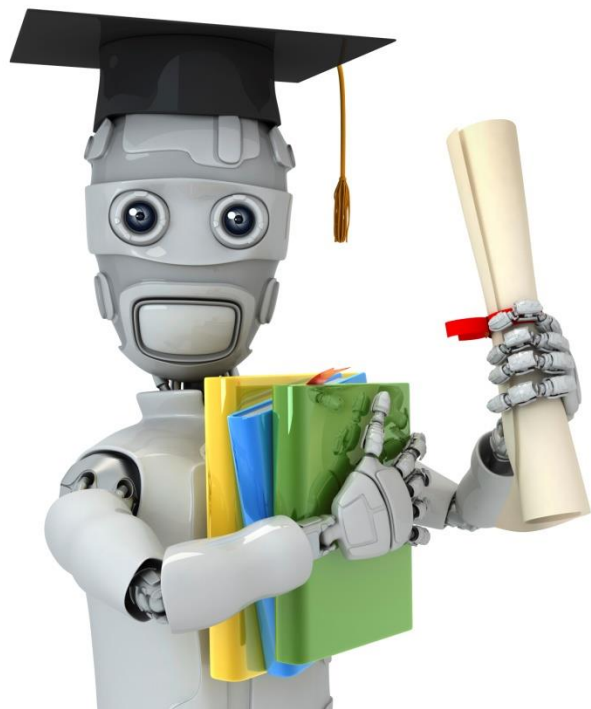
A. `reshape(thetaVec(16:39),4,6)`

B. `reshape(thetaVec(15:38),4,6)`

C. `reshape(thetaVec(16:24),4,6)`

D. `reshape(thetaVec(15:39),4,6)`

E. `reshape(thetaVec(16:39),6,4)`

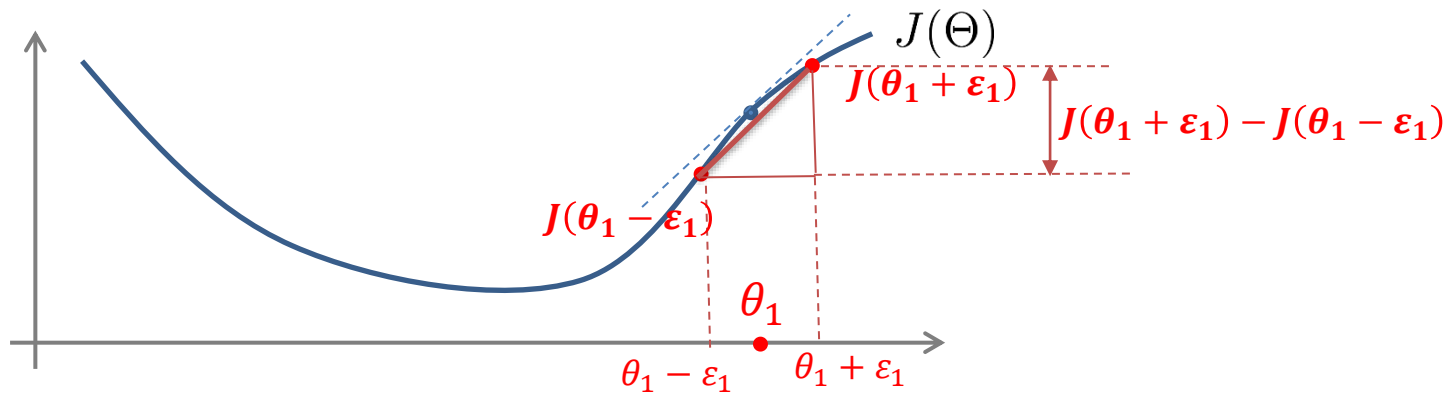


机器学习

神经网络：学习

梯度检查

梯度的数值检验



$$\frac{\partial}{\partial \theta_1} = \frac{J(\theta_1 + \varepsilon_1) - J(\theta_1 - \varepsilon_1)}{2\varepsilon}$$

$$\varepsilon > 0$$

参数向量 θ

$\theta \in \mathbb{R}^n$ (例如 θ 是 $\Theta^{(1)}, \Theta^{(2)}, \Theta^{(3)}$ 的展开版本)

$$\theta = \theta_1, \theta_2, \theta_3, \dots, \theta_n$$

$$\frac{\partial}{\partial \theta_1} J(\theta) \approx \frac{J(\theta_1 + \epsilon, \theta_2, \theta_3, \dots, \theta_n) - J(\theta_1 - \epsilon, \theta_2, \theta_3, \dots, \theta_n)}{2\epsilon}$$

$$\frac{\partial}{\partial \theta_2} J(\theta) \approx \frac{J(\theta_1, \theta_2 + \epsilon, \theta_3, \dots, \theta_n) - J(\theta_1, \theta_2 - \epsilon, \theta_3, \dots, \theta_n)}{2\epsilon}$$

$\vdots$

$$\frac{\partial}{\partial \theta_n} J(\theta) \approx \frac{J(\theta_1, \theta_2, \theta_3, \dots, \theta_n + \epsilon) - J(\theta_1, \theta_2, \theta_3, \dots, \theta_n - \epsilon)}{2\epsilon}$$

```
for i = 1:n,  
    thetaPlus = theta;  
    thetaPlus(i) = thetaPlus(i) + EPSILON;  
    thetaMinus = theta;  
    thetaMinus(i) = thetaMinus(i) - EPSILON;  
    gradApprox(i) = (J(thetaPlus) - J(thetaMinus))  
                    / (2*EPSILON);  
end;
```

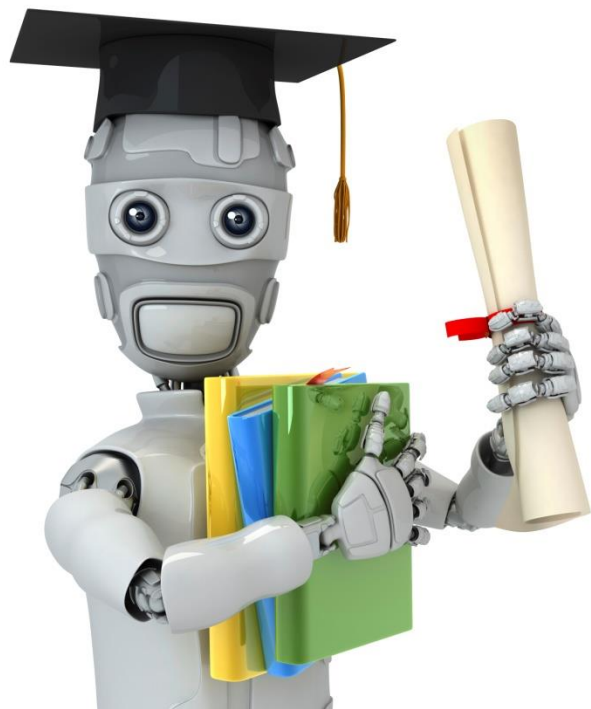
检查 $\text{gradApprox} \approx \text{DVec}$

实现说明:

- 执行反向传播来计算 **DVec** (展开 $D^{(1)}, D^{(2)}, D^{(3)}$)。
- 执行梯度数值检验, 计算 **gradApprox**。
- 确保它们得出相似的值。
- 关闭梯度检查。使用反向传播进行学习。

重要事项:

- 请确保在训练分类器之前禁用梯度检查代码。如果在梯度下降的每次迭代中（或在 **costFunction(...)** 的内循环中）都运行梯度数值计算, 代码的运行速度会非常慢。



机器学习

神经网络：学习

随机初始化

Θ 的初始值

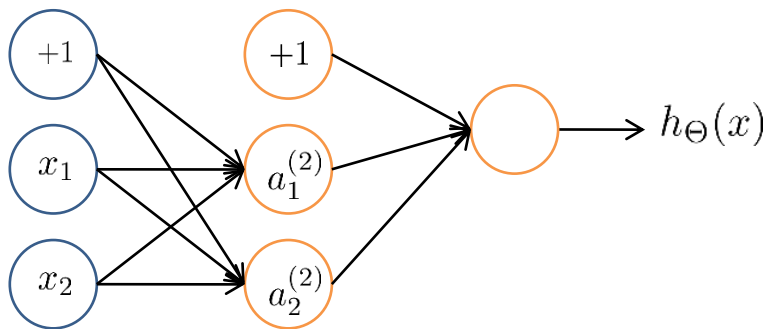
对于梯度下降法和高级优化法，需要初始化 Θ 。

```
optTheta = fminunc(@costFunction,  
                    initialTheta, options)
```

考虑梯度下降

设置 `initialTheta = zeros(n,1)` ?

零初始化



$$\Theta_{ij}^{(l)} = 0 \text{ for all } i, j, l.$$

每次更新后，进入两个隐藏单元的输入所对应的参数都相同。

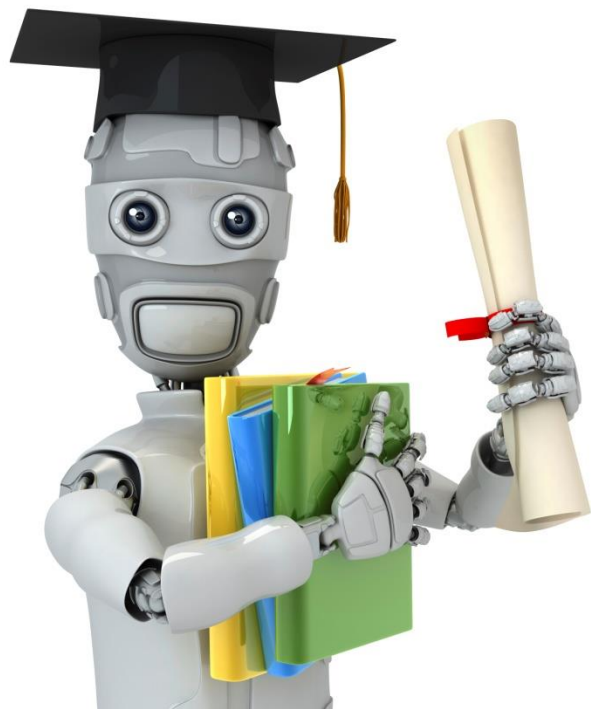
随机初始化：破坏对称性

初始化每个 $\Theta_{ij}^{(l)}$ 为 $[-\epsilon, \epsilon]$ 中的一个随机值
(即 $-\epsilon \leq \Theta_{ij}^{(l)} \leq \epsilon$)

例如

```
Theta1 = rand(10,11) * (2*INIT_EPSILON)
        - INIT_EPSILON;
```

```
Theta2 = rand(1,11) * (2*INIT_EPSILON)
        - INIT_EPSILON;
```



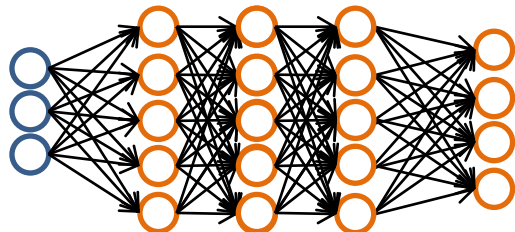
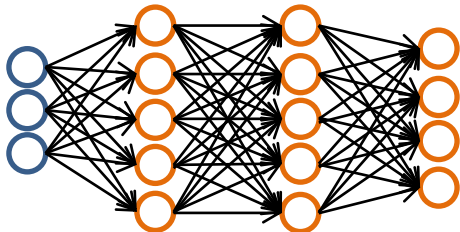
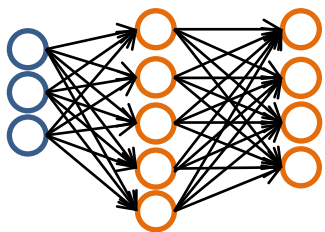
机器学习

神经网络：学习

综合起来

训练一个神经网络

选择网络架构（神经元之间的连接模式）



输入单元数量：特征维度 $x^{(i)}$

输出单元数量：类别数

合理的默认值：1个隐藏层，或者隐藏层 >1 ，则每层的隐藏单元数相同（通常越多越好）。

训练一个神经网络

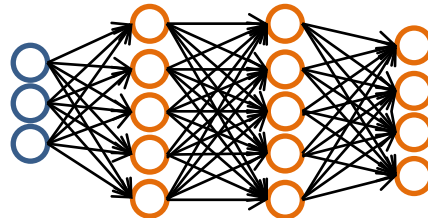
1. 随机初始化权重
2. 执行前向传播，对于 $x^{(i)}$ 计算所有的 $h_{\Theta}(x^{(i)})$
3. 执行代码以计算代价函数 $J(\Theta)$
4. 执行反向传播来计算偏导数 $\frac{\partial}{\partial \Theta_{jk}^{(l)}} J(\Theta)$

for i = 1:m

 执行前向传播和反向传播，使用

 例子 $(x^{(i)}, y^{(i)})$

 (获取 $l = 2, \dots, L$ 时的激活值 $a^{(l)}$ 和误差项 $\delta^{(l)}$).



训练一个神经网络

5. 使用梯度检查来比较使用反向传播计算的 $\frac{\partial}{\partial \Theta_{jk}^{(l)}} J(\Theta)$ 与使用 $J(\Theta)$ 梯度的数值估计。
然后禁用梯度检查代码。
6. 使用梯度下降或带有反向传播的高级优化方法，试图将 $J(\Theta)$ 作为参数 Θ 的函数最小化。

